

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Tue 3 JULY 2018 - Part Theory

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

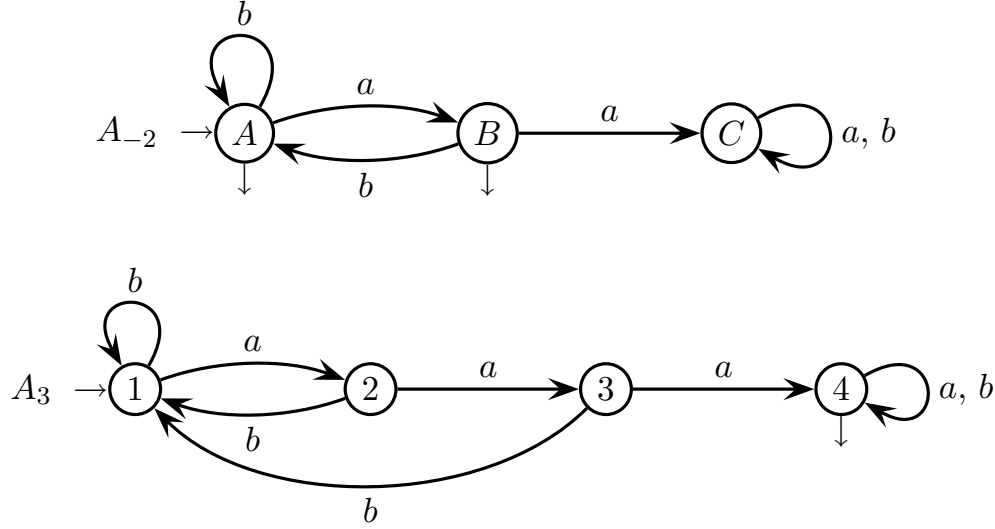
SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider two automata A_{-2} and A_3 over the two-letter alphabet $\Sigma = \{ a, b \}$:



Language $L(A_{-2})$ includes all the strings without any occurrence of a (sub)string aa , whereas language $L(A_3)$ includes all the strings with at least one occurrence of a (sub)string aaa .

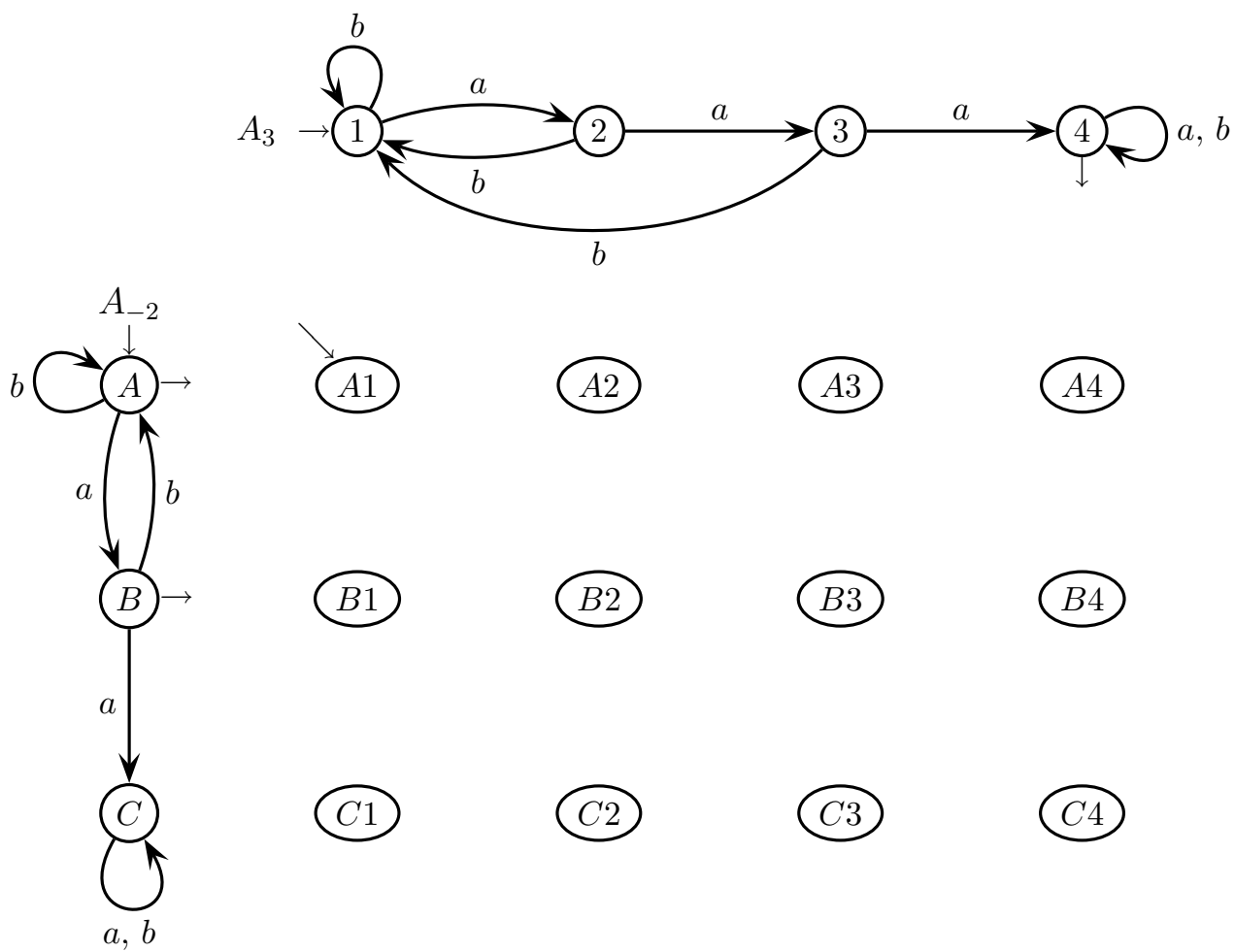
Answer the following questions:

- (a) Transform automaton A_{-2} into a right unilinear grammar G_{-2} .
- (b) Transform grammar G_{-2} into a set of language equations and solve them, thus obtaining a regular expression e_{-2} for the language $L(A_{-2})$.
- (c) Apply the Berry-Sethi method to the regular expression e_{-2} obtained before and construct a deterministic automaton A'_{-2} . Verify that the automaton A'_{-2} thus obtained is identical or equivalent to automaton A_{-2} (if the two automata are not identical, argue that they are equivalent).
- (d) (optional) Apply a variant of the cartesian product automaton construction, usually employed to construct an automaton $A_{\cap}$ that accepts an intersection language, to construct instead an automaton $A_{\cup}$ that accepts the *union language* $L_{\cup} = L_{-2} \cup L_3$. On the grid prepared on the next pages, which is the cartesian product of the state sets of automata A_{-2} and A_3 , first identify the final states of the union automaton $A_{\cup}$ and mark them with an exit arrow, then draw the suitable necessary transitions. Show that:

$$baab \notin L_{\cup} \quad \text{and} \quad baaab \in L_{\cup}$$

by writing the runs of $A_{\cup}$ that have these two strings as input.

grid to be used for question (d)



2 Free Grammars and Pushdown Automata 20%

1. Consider the Dyck language L with two types of parenthesis pairs (open closed): ab and cd . Thus the alphabet is $\Sigma = \{a, b, c, d\}$. The standard (non-ambiguous) grammar that generates language L is:

$$S \rightarrow aSbS \mid cSdS \mid \varepsilon$$

Answer the following questions:

- (a) Suppose to restrict language L and to *admit* only the strings where between any matching pair cd no characters cd can appear (only characters ab are allowed). The restricted language is $L_1 \subset L$.

Examples of Dyck strings that are not admitted in the language L_1 :

$$ccdd \quad accddb \quad cacdcd$$

These instead are *valid* strings for language L_1 :

$$\varepsilon \quad abcd \quad acdbab \quad acdcdb \quad acabdbcd$$

Write a *BNF* grammar G_1 , not ambiguous, that generates language L_1 , and check grammar G_1 by drawing the syntax tree of the sample string:

$$acabdbcd$$

- (b) Suppose to restrict language L and to *admit* only the strings that satisfy these two conditions:
 - the string does not have any matching pair ab or cd , at any depth level, that immediately contains more than one pair ab
 - among the matching pairs at the top level of the string, there is no more than one pair ab

The restricted language is $L_2 \subset L$.

Examples of Dyck strings that are not admitted in the language L_2 :

$$abab \quad cababd \quad abcdaabb$$

These instead are *valid* strings for language L_2 :

$$\varepsilon \quad abcabd \quad cabdaabb$$

Write a *BNF* grammar G_2 , not ambiguous, that generates language L_2 , and check grammar G_2 by drawing the syntax tree of the sample string:

$$cabdaabb$$

- (c) (optional) Countercheck grammars G_1 and G_2 by showing that it is impossible to draw the complete trees of the counterexample strings $ccdd$ and $abab$, respectively. Motivate your answer.

2. We introduce a language for defining, in a general and flexible way, structured data for any data intensive application. The data structure is specified as follows:

- A language phrase consists of one *object*. An object is a non-empty list of *pairs*, separated by a comma “,” and enclosed within braces “{” and “}”.
- A pair consists of a *string* and a *value*, separated by a colon “:”.
- A value is either a string or an object or an *array* or a *number* or a *boolean*.
- An array is a possibly empty list of values, enclosed within square brackets “[” and “]”, and separated by a comma “,”.
- A string consists of a possibly empty sequence of characters surrounded by double quotes, e.g., “color”.
- Individual characters should be modeled in the required grammar by the terminal symbol `char`. Similarly, numbers and booleans should be respectively represented by the terminals `number` and `bool`.

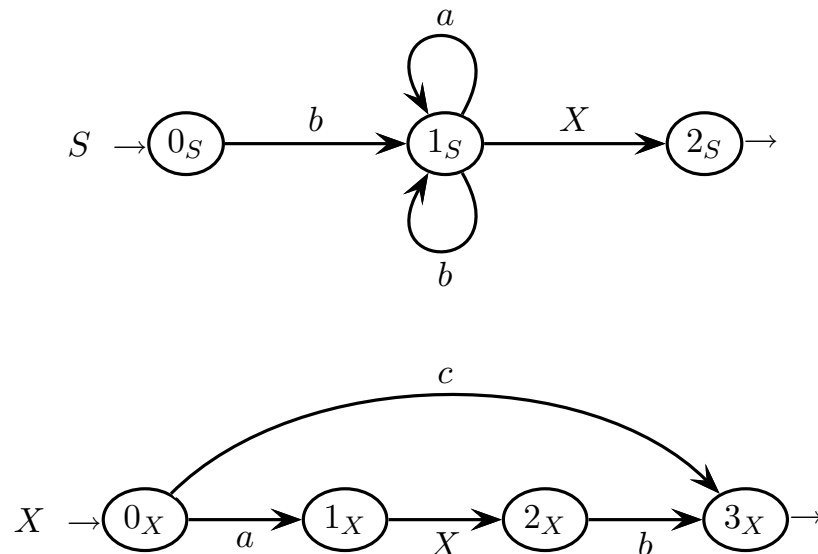
Here is a sample phrase, which does not necessarily exhibit all the above constructs:

```
{
  "colors": [
    {
      "color": "red",
      "category": "hue",
      "type": "primary",
      "code": {
        "rgba": [255, 0, 0, 1],
        "hex": "#FF0"
      }
    },
    {
      "color": "green",
      "category": "hue",
      "type": "secondary",
      "code": {
        "rgba": [0, 255, 0, 1],
        "hex": "#0F0"
      }
    }
  ]
}
```

Write a non-ambiguous grammar, in general of type *EBNF*, that models the language described above.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G , represented as a machine net over the three-letter terminal alphabet $\Sigma = \{ a, b, c \}$ and the two-letter nonterminal alphabet $V = \{ S, X \}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Draw the complete pilot of grammar G and prove that grammar G is of type $ELR(1)$. In particular, examine whether there are multiple transitions and convergent transitions.
- (b) Draw all the guide sets on the net (shift arcs, call arcs and exit arrows), determine whether grammar G is of type $ELL(1)$ and justify your answer.
- (c) Use the Earley algorithm and analyze the sample valid string:

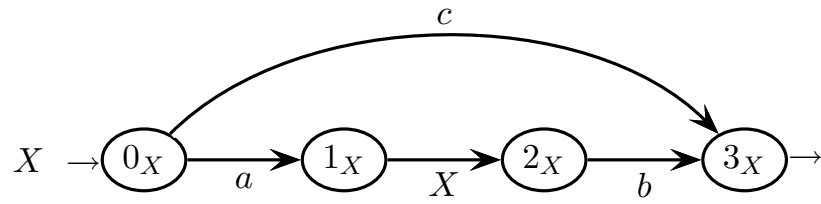
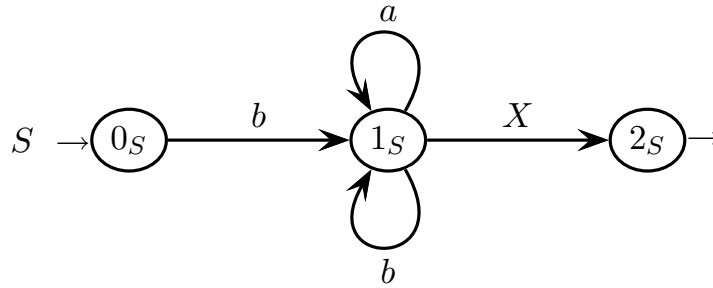
$b a c b$

Draw the syntax tree and show which paths the net machines use to recognize.

- (d) (optional) Suppose that nonterminal X is nullable and modify machine M_X as little as possible. Does anything change in the $ELR(1)$ analysis? For what reasons? Explain what and why.

here draw the pilot of grammar G – question (a)

here draw the call arcs and write all the guide sets of grammar G – question (b)



(the number of rows is not significant)

[illegible]

space for finishing question (c) and answering question (d)

free space for continuation or minutes of any exercise

4 Language Translation and Semantic Analysis 20%

1. Answer the following questions:

- (a) Consider a source language consisting of a fragment of a programming language that includes conditional and assignment instructions, as modeled by the following grammar G_1 (axiom I):

$$G_1 \left\{ \begin{array}{l} I \rightarrow a \mid C \\ C \rightarrow \text{if } E \text{ then } I \text{ else } I \text{ fi} \\ E \rightarrow \text{ce} \end{array} \right.$$

where terminals a and ce represent assignment statements and conditional expressions, respectively.

Write a *BNF* translation scheme or grammar (without changing the source grammar) that defines the translation function τ_1 exemplified below:

$$\tau_1 (\text{if ce then if ce then } a \text{ else } a \text{ fi else } a \text{ fi}) = \\ \text{if (ce) if (ce) } a \text{ other } a \text{ endif other } a \text{ endif}$$

Verify the correctness of the scheme (or grammar) by drawing the translation tree of the example string.

- (b) Now consider another grammar G_2 that models a conditional instruction (non-terminal C , which is also the axiom) that includes a non-empty list of alternatives (nonterminal L):

$$G_2 \left\{ \begin{array}{l} C \rightarrow \text{if } E \text{ then } a \text{ else } L \text{ fi} \\ E \rightarrow \text{ce} \\ L \rightarrow \text{case } E : a ; L \\ L \rightarrow \text{orelse } a \end{array} \right.$$

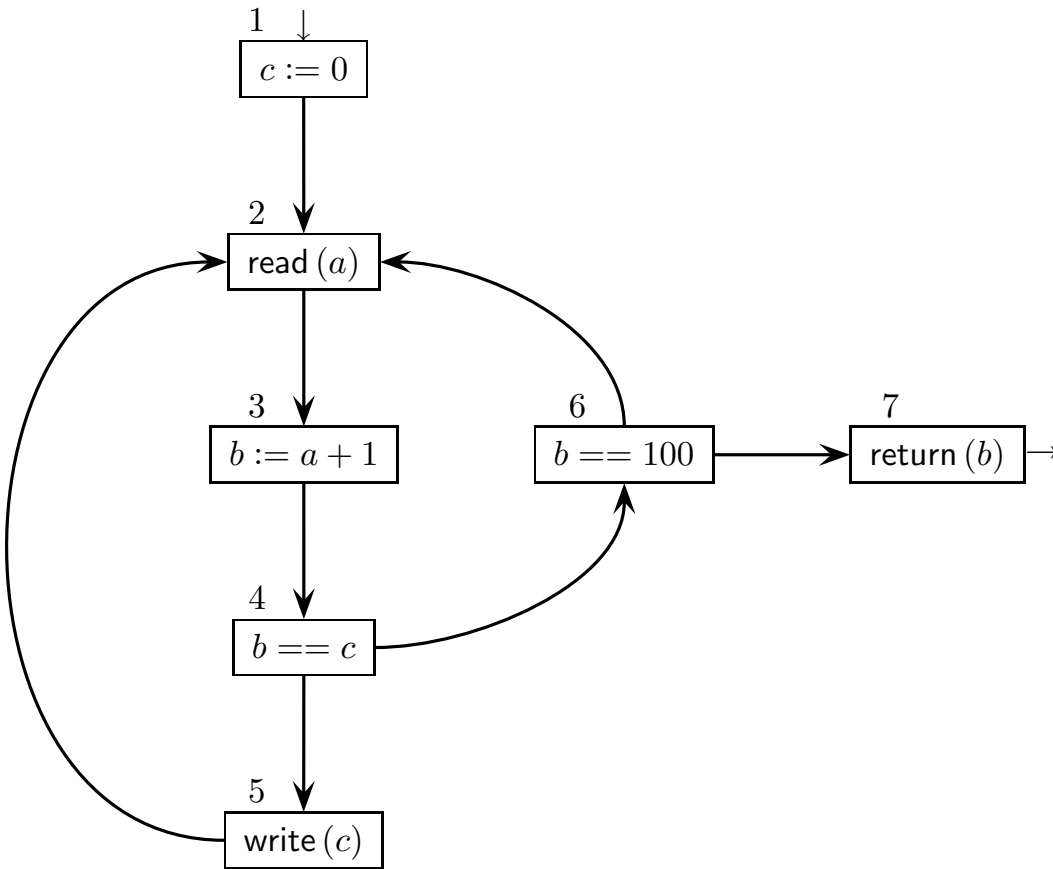
Write a *BNF* translation scheme or grammar (without changing the source grammar) that defines the translation function τ_2 exemplified below:

$$\tau_2 (\text{if ce then } a \text{ else case ce : } a ; \text{ case ce : } a ; \text{ orelse } a \text{ fi}) = \\ \text{if (ce) } a \text{ alt (ce) } a \text{ alt (ce) } a \text{ final } a \text{ endif}$$

Verify the correctness of the scheme (or grammar) by drawing the translation tree of the example string.

- (c) (optional) For each of the two translation schemes (or grammars) defined above, say if it can be defined by means of a finite translation model (a regular translation expression or a finite transducer). Suitably justify your answer; in particular, if a finite translation model exists, then design it.

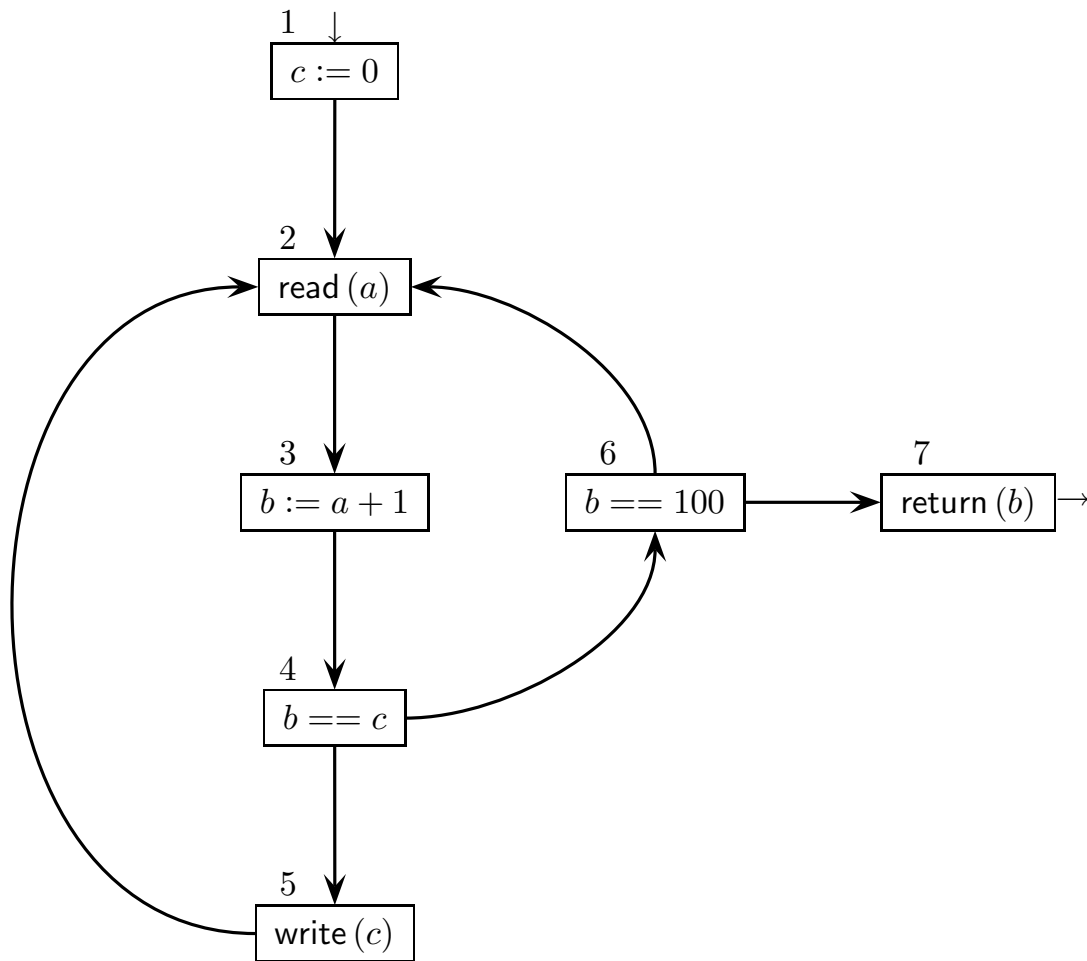
2. Consider the program Control Flow Graph (*CFG*) below, with seven nodes:



Answer the following questions (use the tables / trees / spaces on the next pages):

- (a) Informally find the live variables at the input of each node of the *CFG*.
 - (b) Can variables a and b share the same memory cell (or processor register)? Explain why or why not.
 - (c) (optional) Find again the live variables at the input of each *CFG* node, through the data-flow equation method. Verify that the result is coherent with point (a).
-

decorate the program *CFG* with the live variables found informally – question (a)



space for answering question (b)

space for answering question (c)

tables of definitions and usages at the nodes

<i>node</i>	<i>defined</i>	<i>node</i>	<i>used</i>
1		1	
2		2	
3		3	
4		4	
5		5	
6		6	
7		7	

system of data-flow equations

<i>node</i>	<i>in equations</i>	<i>out equations</i>
1		
2		
3		
4		
5		
6		
7		

iterative solution table of the system of data-flow equations
 (the number of columns is not significant and is more than sufficient to solve the equation system)

	<i>initialization</i>		1		2		3		4		5	
#	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>
1												
2												
3												
4												
5												
6												
7												

	6		7		8		9		10		11	
#	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>
1												
2												
3												
4												
5												
6												
7												

free space for continuation or minutes of any exercise